Docker Interview Questions & Answers

A DevOps Engineer's preparation handbook · Oct 12, 2026 · @MUSTA

How to use this handbook

This handbook answers all 42 unique questions from the source list of 50, in the original order. Eight repeated or overlapping questions were merged into their first occurrence; the appendix shows exactly which ones.

Each answer follows the same pattern:

- **Short answer** — your first 2–4 sentences in the interview. Say this first.
- **Explanation** — the concept in plain words.
- **Hands-on experience** — how you used it in a real project. Speak this part in first person.
- **Example** — a Dockerfile, command or configuration.
- **Interview tip** — what the interviewer is really checking.

Simple questions skip sections they don't need.

The reference project used in the answers

The hands-on answers use the same project as the Jenkins handbook, so your stories stay consistent:

- **Applications:** Java Spring Boot microservices (Maven), one Python Flask API, and a React frontend served by Nginx.
- **Images:** multi-stage Dockerfiles, slim or distroless base images, non-root users.
- **CI/CD:** GitHub → Jenkins → Maven build → SonarQube → Docker build → Trivy scan → push to AWS ECR / Nexus → GitOps update → ArgoCD deploys to EKS.
- **Local development:** Docker Compose with the app, PostgreSQL and Redis.
- **Commands:** written for Docker Engine on Linux with the `docker compose` (v2) plugin.

> **Tip for the whole interview:** Docker questions often lead to Kubernetes questions. When you answer, connect Docker concepts (images, limits, health checks) to how they run in Kubernetes in your project.

Part 1 — Docker Fundamentals

1. What is Docker?

Short answer: Docker is a containerization platform. It packages an application with everything it needs — code, runtime, libraries and configuration — into an image. That image runs the same way as a container on a laptop, a Jenkins agent or a Kubernetes cluster. In my project, every microservice was shipped as a Docker image.

Why we use it

- **"Works on my machine" problem solved** — same image in Dev, QA and Prod.
- **Lightweight** — containers share the host kernel, so they start in seconds and use less memory than VMs.
- **Isolation** — each app has its own dependencies; Java 17 and Java 21 apps can run on the same host.
- **Fast CI/CD** — build once, push to a registry, deploy anywhere.
- **Foundation for Kubernetes** — Kubernetes runs container images.

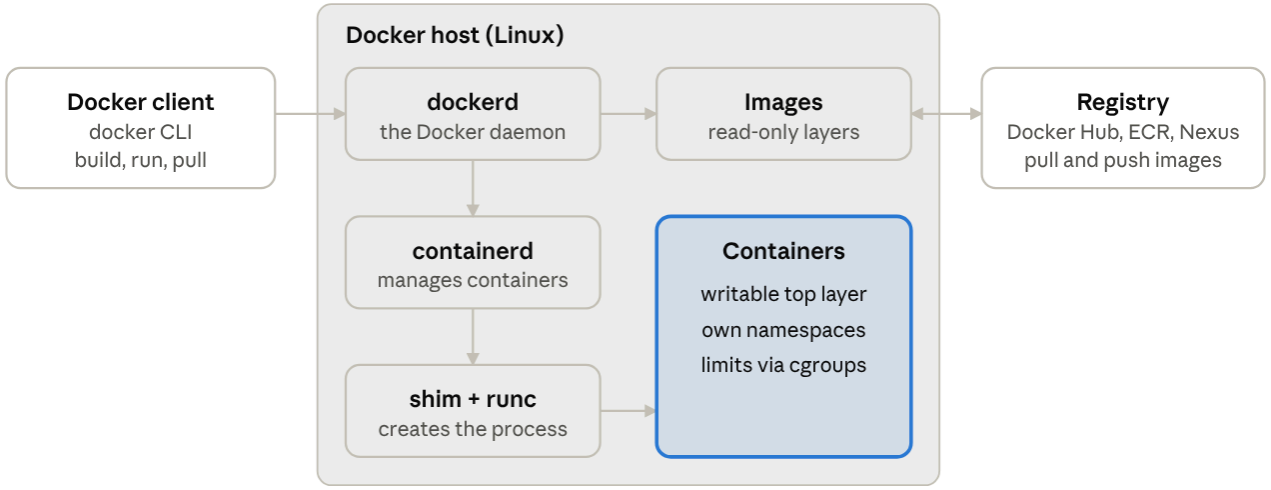
Hands-on experience: In my project, Jenkins built a Docker image for each Spring Boot service, scanned it with Trivy, pushed it to ECR, and ArgoCD deployed the same image to Dev, QA and Prod. Developers used Docker Compose locally to run the service with PostgreSQL and Redis.

> **Interview tip:** Give a one-line definition, then immediately say how *you* used Docker. Don't spend time on history.

2. Explain Docker architecture.

(Also covers source Q18: "What is Docker and explain its architecture?")

Short answer: Docker uses a client-server architecture. The **Docker client** (the `docker` CLI) sends commands to the **Docker daemon** (`dockerd`) through a REST API. The daemon builds images, manages containers, networks and volumes, and pulls or pushes images from a **registry**. Underneath, `containerd` and `runc` actually create and run the containers.



Docker architecture · client, daemon, runtime, registry

The client can be on the same machine or remote; the daemon does the work, and runc uses Linux namespaces and cgroups to start each container.

Components

- **Docker client** — the CLI. `docker build`, `docker run`, `docker pull` are API calls to the daemon.
- **Docker daemon (`dockerd`)** — listens on `/var/run/docker.sock`; manages images, containers, networks, volumes.
- **containerd** — container runtime that manages the container lifecycle (also used directly by Kubernetes).
- **containerd-shim + runc** — runc creates the container process with namespaces and cgroups; the shim keeps it running even if the daemon restarts.
- **Docker objects** — images, containers, volumes, networks.
- **Registry** — stores images: Docker Hub, AWS ECR, Nexus, Harbor.

Hands-on experience: On our Jenkins agents, the Jenkins user ran `docker build` and `docker push`; the CLI talked to the local daemon through `/var/run/docker.sock`. That's also why we treated access to the socket as root-level access and limited which jobs could use those agents.

> **Interview tip:** Mention containerd and runc. It shows you know Docker is more than one program — and why Kubernetes can run without Docker but still run Docker-built images.

3. What is the difference between a Docker image and a Docker container?

Short answer: An image is a read-only template — the packaged application with its filesystem and settings. A container is a running instance of that image. One image can start many containers, and each container gets its own writable layer on top of the image.

	Image	Container
What it is	Read-only template (blueprint)	Running (or stopped) instance
Created by	`docker build` or `docker pull`	`docker run` / `docker create`
Changes	Immutable; a change means a new image	Has a writable layer; changes are lost when removed
Stored in	Registry and local image store	Only on the host where it runs
Analogy	Class	Object

Example

```
docker pull nginx:stable-alpine          # get the image
docker run -d --name web1 nginx:stable-alpine
docker run -d --name web2 nginx:stable-alpine   # two containers, one image
docker images                            # list images
docker ps                                # list running containers
```

> **Interview tip:** Say "the container's writable layer is temporary — that's why we use volumes for data." It connects nicely to the volumes question.

4. What is the difference between COPY and ADD?

(Also covers source Q14.)

Short answer: Both copy files into the image. `COPY` simply copies local files and folders. `ADD` can also download from a URL and automatically extracts local tar archives. Best practice is to use `COPY` by default, because it is clear and predictable, and use `ADD` only when you really need auto-extraction.

Explanation

	COPY	ADD
Local files/folders	Yes	Yes

	COPY	ADD
Remote URL	No	Yes (downloads, does not extract)
Auto-extract local `.tar`, `.tar.gz`	No	Yes
Recommended	Yes, by default	Only for special cases

- For downloading files, `RUN curl ... && verify checksum` gives more control, or `ADD --checksum=sha256:...` with BuildKit.
- Both support `--chown` to set file ownership, which helps when running as a non-root user.

Example

```
COPY target/app.jar /app/app.jar          # normal copy
ADD rootfs.tar.gz /                       # auto-extracts the archive
COPY --chown=app:app config/ /app/config/ # copy with ownership
```

Hands-on experience: In our Dockerfiles we used only `COPY`. Once, a Dockerfile used `ADD` for a `.tar.gz` that was meant to be copied as-is, and Docker extracted it unexpectedly. Switching to `COPY` fixed it.

> **Interview tip:** The expected answer is "prefer COPY" plus the two extra features of ADD.

5. What is the difference between CMD and ENTRYPOINT?

(Also covers source Q15.)

Short answer: Both define what runs when the container starts. `ENTRYPOINT` sets the main executable, which is not replaced by arguments in `docker run`. `CMD` provides default arguments or a default command, which is easily overridden. A common pattern is `ENTRYPOINT` for the program and `CMD` for its default arguments.

Explanation

	CMD	ENTRYPOINT
Purpose	Default command or default args	Fixed main executable
`docker run image xyz`	`xyz` **replaces** CMD	`xyz` is **appended** as arguments

	CMD	ENTRYPOINT
Override	Just add a command	`docker run --entrypoint ...`
Combined	CMD becomes default args to ENTRYPOINT	—

- **Use exec form** (`["java", "-jar", "app.jar"]`), not shell form (`java -jar app.jar`). With exec form, the app runs as PID 1 and receives `SIGTERM` on `docker stop` , so it shuts down gracefully.
- If there are multiple `CMD` or `ENTRYPOINT` lines, only the last one counts.

Example

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
docker run orders:1.0                                  # runs with prod
profile
docker run orders:1.0 --spring.profiles.active=dev     # replaces only the
CMD part
```

Hands-on experience: One service took 10 seconds to stop and was killed instead of shutting down cleanly. Its Dockerfile used shell form, so `/bin/sh` was PID 1 and didn't pass `SIGTERM` to Java. Changing to exec form fixed graceful shutdown during Kubernetes rolling updates.

> **Interview tip:** Mention exec form and signal handling. It's the detail that shows real production experience.

6. How do you reduce Docker image size?

(Also covers source Q13.)

Short answer: I use small base images like `-slim` , Alpine or distroless, multi-stage builds so build tools don't go into the final image, a `.dockerignore` file, fewer and cleaner layers, and I remove package caches in the same `RUN` step that creates them. In my project, this brought a Java image from about 700 MB down to around 200 MB.

Techniques

1. **Smaller base image** — `eclipse-temurin:17-jre` instead of a full JDK; `python:3.12-slim` instead of `python:3.12` ; distroless or Alpine where suitable.

2. **Multi-stage builds** — build with Maven/Node in one stage, copy only the result to a runtime stage (Q7).
3. **`.dockerignore`** — exclude `.git`, `target/`, `node_modules/`, logs and local env files.
4. **Clean up in the same layer** — a file deleted in a later layer still exists in the earlier layer.
5. **Install only what's needed** — `--no-install-recommends`, `pip install --no-cache-dir`, `npm ci --omit=dev`.
6. **Combine related `RUN` commands** to avoid extra layers.
7. **Check what is big** — `docker history <image>` or the `dive` tool.

Example

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

```
# .dockerignore
.git
__pycache__/
*.log
.env
tests/
```

Hands-on experience: Our Python API image was 1.1 GB because it used the full `python` image and copied the whole repo, including `.git`. Switching to `-slim`, adding `.dockerignore` and `--no-cache-dir` brought it to about 180 MB. Smaller images pulled faster on EKS nodes and had fewer vulnerabilities in Trivy scans.

> **Interview tip:** Connect size to benefits: faster pulls, faster scaling, and fewer CVEs. That shows you think beyond just megabytes.

7. What is a multi-stage Docker build, and how does it reduce image size?

(Also covers source Q32: "How do you reduce image size using multi-stage builds?")

Short answer: A multi-stage build uses more than one `FROM` in a Dockerfile. The first stage has the build tools — Maven, JDK, npm — and builds the app. The final stage starts from a small runtime image and copies only the built artifact using `COPY --from`. Build tools, source code and caches never reach the final image, so it is smaller and more secure.

Example (Java)

```
# Stage 1: build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B package -DskipTests

# Stage 2: runtime
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
USER 1000
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

- `AS build` names the first stage.
- `COPY --from=build` copies just the JAR from the build stage.
- The final image has only the JRE and the JAR — no Maven, no source, no `.m2` cache.
- `docker build --target build .` builds only the first stage, useful for running tests.

Size impact: a single-stage image with Maven + JDK + source is often 700–800 MB; the multi-stage runtime image is usually around 200–300 MB for a Spring Boot app.

Hands-on experience: We used multi-stage builds for all Java and React services. For React, stage one ran `npm ci && npm run build`, and stage two was `nginx` serving only the static `build/` folder — around 50 MB instead of more than 1 GB with `node_modules`.

> **Interview tip:** Add the security angle: fewer tools in the final image means a smaller attack surface.

8. What is Docker Compose?

Short answer: Docker Compose is a tool to define and run multi-container applications using one YAML file, `compose.yaml`. With one command, `docker compose up`, it creates all the services, networks and volumes. We used it mainly for local development and testing, not for production orchestration.

Explanation

- **Services** — each container (app, database, cache).

- **Networks** — Compose creates a network, so services reach each other by name (`db:5432`).
- **Volumes** — persistent data for databases.
- **Commands** — `docker compose up -d`, `docker compose ps`, `docker compose logs -f`, `docker compose down`.
- Modern Compose is the `docker compose` plugin (v2); the old `docker-compose` v1 binary is deprecated, and the top-level `version:` field is no longer needed.

Example

```
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/orders
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: orders
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10
volumes:
  pgdata:
secrets:
  db_password:
    file: ./db_password.txt
```

- `depends_on` with `service_healthy` waits until Postgres is actually ready, not just started.
- The app reaches the database at hostname `db`.

Hands-on experience: New developers ran `docker compose up` and had the service, Postgres and Redis running in two minutes. In Jenkins, we also used Compose to start dependencies for integration tests, then `docker compose down -v` to clean up.

> **Interview tip:** Say clearly "Compose for local and testing, Kubernetes for production." Interviewers often check that you know the difference.

9. What is Docker Swarm?

Short answer: Docker Swarm is Docker's built-in container orchestration. It joins multiple Docker hosts into a cluster and runs **services** with a set number of replicas, load balancing, rolling updates and self-healing. It's simpler than Kubernetes, but most companies today, including mine, use Kubernetes for production.

Explanation

- **Manager nodes** manage the cluster state (Raft consensus); **worker nodes** run tasks.
- **Service** = desired state, for example 3 replicas of `nginx`.
- **Overlay networks** connect containers across hosts.
- **Routing mesh** — a published port works on every node.
- **Rolling updates and rollback** — `docker service update --image ...`, `docker service rollback`.

Example

```
docker swarm init --advertise-addr 10.0.1.10
docker service create --name web --replicas 3 -p 80:80 nginx:stable-alpine
docker service scale web=5
docker service update --image nginx:stable-alpine --update-parallelism 1 web
```

	Docker Swarm	Kubernetes
Setup	Very simple, built into Docker	More complex
Features	Basic orchestration	Autoscaling, rich ecosystem, operators, RBAC
Usage	Small setups	Industry standard

> **Interview tip:** If you haven't used Swarm in production, say so honestly and explain why your team chose Kubernetes. Honesty works better than pretending.

10. What are Docker volumes?

(Also covers source Q47: "What is a Docker volume?")

Short answer: A volume is storage managed by Docker that lives outside the container's writable layer. Data in a volume survives when the container is removed or replaced. We use volumes for databases, uploaded files and anything that must persist. Docker also supports bind mounts (a host folder) and tmpfs (memory only).

Types of mounts

Type	Where data lives	Use case
Named volume	`/var/lib/docker/volumes/<name>/_data`, managed by Docker	Databases, persistent app data
Bind mount	Any host path you choose	Local development (live code reload), config files
tmpfs	Host memory only	Temporary or sensitive data

Commands

```
docker volume create pgdata
docker volume ls
docker volume inspect pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16
docker volume prune        # remove unused volumes (careful!)
```

Hands-on experience: Locally, our Postgres container used a named volume, so developers kept their test data across `docker compose down` and `up`. In Kubernetes, the same need was handled by PersistentVolumeClaims backed by EBS.

> **Interview tip:** Mention that data in the container's writable layer is lost when the container is removed — that's the "why" behind volumes.

11. What does `EXPOSE 8080` do in a Dockerfile?

Short answer: `EXPOSE 8080` only documents that the application listens on port 8080. It does **not** publish the port or make it reachable from outside. To access it from the host, we still need `-p 8080:8080` in `docker run`. `docker run -P` publishes all exposed ports to random host ports.

Example

```
EXPOSE 8080
```

```
docker run -d -p 8080:8080 orders:1.0     # host 8080 → container 8080
docker run -d -P orders:1.0               # random host port
docker port <container>                   # see the mapping
```

- Containers on the same Docker network can reach port 8080 even without `EXPOSE` or `-p`.
- In Kubernetes, `EXPOSE` is ignored; the port is defined in the Pod spec and Service.

> **Interview tip:** The key line is "EXPOSE is documentation; `-p` publishes." Many candidates get this wrong.

12. How do you get a running container's IP address?

Short answer: I use `docker inspect` with a format filter: `docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' <container>`. In practice, though, we connect containers by **name** on a user-defined network, because container IPs change every time the container is recreated.

Commands

```
# IP address on all networks the container is attached to
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
web1

# Full network details
docker inspect web1 | grep -i ipaddress

# All containers on a network with their IPs
docker network inspect my-app-net

# From inside the container (if tools exist)
docker exec web1 hostname -i
```

Hands-on experience: A developer hardcoded a container IP in a config file, and it broke after a restart. We moved the services to a user-defined bridge network and used the container name as hostname. Docker's built-in DNS resolved it every time.

> **Interview tip:** Give the command, then say "don't rely on IPs; use service names." That second sentence is what makes the answer strong.

Part 2 — Dockerfile Basics

13. What is a Dockerfile?

Short answer: A Dockerfile is a text file with step-by-step instructions to build a Docker image: which base image to start from, which files to copy, which commands to run, and how to start the application. `docker build` reads it and creates the image. Because it's a file in Git, image builds are repeatable and reviewable.

Explanation

- Each instruction (`FROM`, `RUN`, `COPY`, `CMD` and so on) is executed in order.
- Instructions that change the filesystem (`RUN`, `COPY`, `ADD`) create image layers, and Docker caches them.
- The Dockerfile lives in the application repo, next to the code and the Jenkinsfile.
- The **build context** (usually `.`) is the folder sent to the builder; `.dockerignore` controls what is excluded.

Hands-on experience: In my project, every service repo had a `Dockerfile` at the root. Jenkins ran `docker build -t <repo>:<commit-sha> .` on it. Changes to the Dockerfile went through pull requests, and we used `hadolint` in the pipeline to catch bad practices like missing version tags.

> **Interview tip:** Mention layers, caching and `.dockerignore` in one sentence. It shows you understand how the Dockerfile is used, not just what it is.

14. Write a Dockerfile for a simple web application.

(Also covers source Q30: "Write a Dockerfile for a simple web application." Nginx and Java examples are in Q24 and Q25.)

Short answer: Here's a production-style Dockerfile for a Python Flask web app. It uses a slim base image, installs dependencies before copying the code for better caching, runs as a non-root user, and starts with Gunicorn in exec form.

Example

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app
```

```
# 1. Dependencies first (cached unless requirements.txt changes)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 2. Application code
COPY . .

# 3. Run as a non-root user
RUN useradd --create-home --uid 10001 appuser
USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "app:app"]
```

What each part does

- `FROM python:3.12-slim` — small official base image.
- `PYTHONUNBUFFERED=1` — logs appear immediately in `docker logs`.
- `COPY requirements.txt` then `pip install` — dependency layer is cached, so code changes rebuild fast.
- `--no-cache-dir` — doesn't keep pip's download cache in the image.
- `USER appuser` — the app doesn't run as root.
- `HEALTHCHECK` — Docker marks the container healthy or unhealthy. (Kubernetes ignores this and uses its own probes.)
- `CMD [...]` — exec form, so Gunicorn gets stop signals directly.
- `0.0.0.0` — listens on all interfaces; `127.0.0.1` would not be reachable from outside the container.

Build and run

```
docker build -t flask-app:1.0 .
docker run -d --name flask-app -p 8000:8000 flask-app:1.0
curl http://localhost:8000/health
```

> **Interview tip:** If asked to write one on the spot, the minimum is `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD`. Then mention the extras — non-root user, caching order, slim image — to stand out.

Part 3 — How Containers Work

15. What is the difference between containers and virtual machines?

Short answer: A virtual machine runs a full guest operating system on top of a hypervisor, so each VM has its own kernel. A container shares the host's kernel and only packages the app and its libraries. That makes containers much lighter and faster to start, while VMs give stronger isolation.

	Virtual machine	Container
Isolation	Hardware-level, via hypervisor	Process-level, via namespaces and cgroups
Kernel	Own guest kernel per VM	Shares host kernel
Size	GBs (full OS)	MBs (app + libraries)
Startup	Minutes	Seconds or less
Density	Fewer per host	Many per host
Security boundary	Stronger	Weaker by default; needs hardening
OS flexibility	Any OS (Windows VM on Linux host)	Must match the host kernel type (Linux containers need a Linux kernel)

Hands-on experience: In my project, we used both together: EKS worker nodes were EC2 virtual machines, and our microservices ran as containers on them. VMs gave us isolation between nodes and accounts; containers gave us fast deployments and high density — around 30 pods per node.

> **Interview tip:** Say "it's not containers *or* VMs — in the cloud, containers usually run on VMs." That shows practical understanding.

16. How do containers share OS resources?

Short answer: All containers on a host share the same Linux kernel. The kernel keeps them separate using **namespaces**, which control what a container can *see* (its own processes, network, filesystem, hostname), and **cgroups**, which control how much it can *use* (CPU, memory, I/O). Files are shared efficiently through image layers with a union filesystem like overlay2.

Explanation

- **Kernel** — one kernel, shared by all containers. No separate OS boot.
- **Namespaces** — isolation of view: PID, network, mount, UTS, IPC, user (Q18).
- **cgroups** — limits and accounting for CPU, memory, PIDs and block I/O (Q17).
- **Union filesystem (overlay2)** — containers from the same image share the read-only image layers; each gets only its own small writable layer (Q19).
- **Capabilities, seccomp, AppArmor/SELinux** — restrict what system calls and privileges a container can use.

Hands-on experience: On one EKS node, ten replicas of the same service shared the same image layers on disk, so the image was stored only once. Each container only added its own small writable layer and its own memory usage.

> **Interview tip:** "Namespaces = what you can see, cgroups = what you can use." That one line is easy to remember and interviewers like it.

17. How does resource allocation work for containers?

Short answer: Docker uses Linux cgroups to limit CPU, memory and other resources per container. By default, a container has **no limits** and can use all host resources. We set limits like `--memory` and `--cpus`. If a container exceeds its memory limit, the kernel kills it (OOM kill, exit code 137). CPU limits only slow the container down; they don't kill it.

Common options

Option	Meaning
`--memory=512m`	Hard memory limit; exceeded → OOM kill
`--memory-reservation=256m`	Soft limit, used when the host is under memory pressure
`--cpus=1.5`	Use at most 1.5 CPU cores' worth of time
`--cpu-shares=512`	Relative weight when CPUs are busy (default 1024)
`--cpuset-cpus=0,1`	Pin to specific cores
`--pids-limit=200`	Max number of processes (protects against fork bombs)

Example

```
docker run -d --name orders --memory=768m --cpus=1 --pids-limit=200
orders:1.0
docker stats orders          # live CPU and memory usage
docker inspect -f '{{.State.OOMKilled}}' orders   # was it OOM-killed?
```

Java detail: modern JVMs are container-aware. Use `-XX:MaxRAMPercentage=75.0` so the heap fits inside the container limit, leaving room for non-heap memory.

Hands-on experience: A Java service kept restarting with exit code 137. `docker inspect` showed `OOMKilled: true`. The JVM was set with a fixed `-Xmx1g` but the container limit was 1 GB, leaving no room for metaspace and threads. Switching to `MaxRAMPercentage=75` fixed it. In Kubernetes, the same values became `resources.requests` and `resources.limits`.

> **Interview tip:** Mention "no limits by default" and "exit code 137 = OOM kill". Both are frequently asked follow-ups.

18. What are namespaces in Docker?

Short answer: Namespaces are a Linux kernel feature that gives each container its own isolated view of the system. Inside a container, the process sees only its own processes, network interfaces, mounts and hostname, as if it were alone on the machine. Docker creates a set of namespaces for every container.

Namespace types

Namespace	Isolates	Effect in a container
PID	Process IDs	App runs as PID 1; can't see host processes
NET	Network stack	Own interfaces, IP, routing, ports
MNT	Mount points	Own root filesystem
UTS	Hostname	Own hostname
IPC	Shared memory, semaphores	Can't use other containers' IPC
USER	User and group IDs	Root in container can map to a non-root user on the host (user namespace remapping, rootless mode)
CGROUP	cgroup view	Sees only its own cgroup tree

Example

```
docker run -d --name web nginx:stable-alpine
docker exec web ps aux             # sees only nginx processes
docker inspect -f '{{.State.Pid}}' web   # host PID of the container's main
process
sudo ls -l /proc/<host-pid>/ns     # namespaces of that process
```

Sharing is also possible: `--network host` or `--pid host` makes the container use the host's namespace — useful for debugging tools, but it reduces isolation.

> **Interview tip:** Name at least five namespaces and give the PID example. Mentioning the user namespace for security is a bonus.

19. What are Docker layers?

Short answer: A Docker image is made of read-only layers. Each instruction that changes the filesystem — `RUN`, `COPY`, `ADD` — adds a new layer on top of the previous one. When a container starts, Docker adds a thin writable layer on top. Layers are cached and shared between images, which makes builds faster and saves disk space.

Key points

- **Union filesystem (overlay2)** stacks the layers into one view.
- **Copy-on-write** — when a container changes a file from an image layer, the file is copied up to the writable layer first.
- **Build cache** — if an instruction and its inputs haven't changed, Docker reuses the cached layer. Once one layer changes, all layers **after** it are rebuilt.
- **Order matters** — put things that change rarely (dependencies) before things that change often (source code).
- **Deleting files later doesn't shrink the image** — the file still exists in the earlier layer.

Example — good vs bad ordering

```
# Bad: any code change re-runs npm install
COPY . .
RUN npm ci

# Good: npm install is cached until package files change
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
```

```
docker history orders:1.0     # see each layer and its size
```

Hands-on experience: Our frontend build took 6 minutes in Jenkins because every commit re-ran `npm ci`. Reordering the Dockerfile to copy `package*.json` first brought rebuilds down to about 1 minute when only code changed.

> **Interview tip:** Explain cache invalidation ("everything after a changed layer rebuilds"). It's the practical heart of this question.

20. What is the Docker container lifecycle?

Short answer: A container goes through these states: **created** → **running** → optionally **paused** → **stopped (exited)** → **removed**. It can also be **restarting** if a restart policy is set. Each state change matches a Docker command.

States and commands

Command	From → To
`docker create`	image → created
`docker start`	created / exited → running
`docker run`	create + start in one step
`docker pause` / `unpause`	running ↔ paused (processes frozen)
`docker stop`	running → exited (SIGTERM, then SIGKILL after 10s by default)
`docker kill`	running → exited (SIGKILL immediately)
`docker restart`	stop + start
`docker rm`	exited → removed (writable layer deleted)

Restart policies: `--restart no | on-failure[:max] | always | unless-stopped`.

Useful exit codes: `0` normal exit, `1` application error, `137` killed by SIGKILL (often OOM), `143` stopped with SIGTERM.

Hands-on experience: For a few services still on plain Docker hosts, we used `--restart unless-stopped` so containers came back after a server reboot. When a container kept going from running to exited, I checked the exit code in `docker ps -a` first — it quickly told me whether it was an app error or an OOM kill.

> **Interview tip:** Explain `stop` vs `kill` (graceful vs immediate). It's a common follow-up.

21. What happens internally when you run `docker run`?

Short answer: The CLI sends the request to the Docker daemon. The daemon checks if the image exists locally, pulls it if not, creates the container with a writable layer, sets up networking and volumes, and then asks containerd and runc to create the namespaces and cgroups and start the process. Finally, it streams logs or returns the container ID.

Step by step for `docker run -d -p 8080:80 --name web nginx`

1. **CLI → API** — the `docker` CLI sends a REST call to `dockerd` over `/var/run/docker.sock`.
2. **Image check** — is `nginx:latest` available locally? If not, pull it layer by layer from the registry.
3. **Create container** — build the container config, add a writable layer on top of the image layers (overlay2).
4. **Network** — create a veth pair, attach it to the `docker0` bridge, assign an IP, and add iptables/NAT rules for `-p 8080:80`.
5. **Volumes** — mount any volumes or bind mounts.
6. **Runtime** — `dockerd` asks `containerd`, which starts a `containerd-shim`, which calls `runc`.
7. **Isolation** — runc creates namespaces (PID, NET, MNT, UTS, IPC), applies cgroup limits, capabilities and seccomp.
8. **Start** — runc starts the `ENTRYPOINT` / `CMD` as PID 1 in the container, then exits; the shim stays as the parent.
9. **Output** — with `-d`, the container ID is printed; logs go to the logging driver (json-file by default).

> **Interview tip:** You don't need every detail. Hitting "pull if missing → writable layer → network → containerd/runc → namespaces and cgroups → PID 1" is a strong answer.

22. Why does using only Docker (without orchestration) cause downtime?

Short answer: Plain Docker manages containers on one host only. To deploy a new version, we usually stop the old container and start a new one, so there's a gap. There's no built-in rolling update, no load balancing across replicas, no health-based traffic switching, and if the host fails, everything on it goes down. Orchestrators like Kubernetes solve these problems.

Why downtime happens

- **Stop-then-start deployments** — the port can't be used by two containers at once, so old must stop before new starts.

- **Single host** — a server reboot or crash takes all containers down.
- **No rolling updates** — nothing replaces containers one by one while keeping others serving.
- **No traffic management** — no Service or load balancer that routes only to healthy containers.
- **Limited self-healing** — restart policies restart a crashed container on the same host, but can't move it elsewhere.
- **No autoscaling** for traffic spikes.

How Kubernetes fixes it: multiple replicas across nodes, rolling updates with `maxUnavailable: 0`, readiness probes so traffic only reaches ready pods, rescheduling when a node fails, and Horizontal Pod Autoscaler.

Hands-on experience: Before Kubernetes, our deployments on EC2 with plain Docker caused about 20–30 seconds of downtime per release, because we ran `docker stop` and `docker run`. After moving to EKS with rolling updates and readiness probes, releases became zero-downtime.

> **Interview tip:** Give the contrast with Kubernetes features in one sentence each. The interviewer wants to see you know *why* orchestration exists.

Part 4 — Writing Dockerfiles & Building Images

23. What are the components (instructions) of a Dockerfile?

Short answer: The main instructions are `FROM` for the base image, `WORKDIR`, `COPY` / `ADD` for files, `RUN` for build commands, `ENV` and `ARG` for variables, `EXPOSE` to document ports, `USER` to set the runtime user, and `CMD` / `ENTRYPOINT` to define what starts. There are also `LABEL`, `VOLUME`, `HEALTHCHECK`, `SHELL` and `STOPSIGNAL`.

Instruction	Purpose	Example
`FROM`	Base image; starts a build stage	`FROM eclipse-temurin:17-jre`
`ARG`	Build-time variable (not in the running container)	`ARG APP_VERSION=1.0`
`ENV`	Environment variable at build and run time	`ENV TZ=Asia/Kolkata`

Instruction	Purpose	Example
`WORKDIR`	Sets and creates the working directory	`WORKDIR /app`
`COPY`	Copy files from build context	`COPY target/app.jar app.jar`
`ADD`	Like COPY + URL download + tar extraction	`ADD app.tar.gz /opt/`
`RUN`	Run a command at build time (new layer)	`RUN apt-get update && apt-get install -y curl`
`EXPOSE`	Document the listening port	`EXPOSE 8080`
`USER`	User for following instructions and runtime	`USER 10001`
`VOLUME`	Mark a path as a mount point	`VOLUME /data`
`LABEL`	Metadata	`LABEL org.opencontainers.image.source="..."`
`HEALTHCHECK`	Command Docker runs to check health	`HEALTHCHECK CMD curl -f http://localhost:8080/health || exit 1`
`ENTRYPOINT`	Main executable	`ENTRYPOINT ["java","-jar","app.jar"]`
`CMD`	Default command or default arguments	`CMD ["--server.port=8080"]`
`STOPSIGNAL`	Signal sent on `docker stop`	`STOPSIGNAL SIGTERM`

- **Never put secrets in `ARG` or `ENV`** — they are visible in `docker history` and `docker inspect`. Use BuildKit secrets (`RUN --mount=type=secret,...`).

> **Interview tip:** Explain `ARG` vs `ENV` (build-time vs runtime). It's the most common follow-up on this question.

24. Write a lightweight Dockerfile for an Nginx web server.

Short answer: I start from an Alpine-based Nginx image, copy only the static site and my Nginx config, and run it as non-root. The unprivileged Nginx image listens on port 8080, so it doesn't need root to bind a low port.

Example

```
FROM nginxinc/nginx-unprivileged:stable-alpine

# Custom server config
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Static website files only
COPY --chown=nginx:nginx ./site/ /usr/share/nginx/html/

EXPOSE 8080
# The base image already runs as the non-root 'nginx' user
# and starts: nginx -g 'daemon off;'
```

`nginx.conf`:

```
server {
    listen 8080;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;   # SPA routing (React/Angular)
    }

    location /health {
        return 200 'ok';
    }
}
```

Why it's lightweight and safe

- Alpine base keeps the image around 50 MB or less.
- Only static files are copied — no build tools.
- Runs as non-root; listens on 8080.
- `try_files ... /index.html` makes front-end routing work.
- In production, pin a specific version or digest instead of `stable-alpine`, so builds are repeatable.

Simpler version (official image, runs as root by default):

```
FROM nginx:stable-alpine
COPY ./site/ /usr/share/nginx/html/
EXPOSE 80
```

Hands-on experience: Our React frontend used a multi-stage build: `node` to run `npm run build`, then the unprivileged Nginx image to serve the `build/` folder. Running as non-root was required by our Kubernetes Pod Security settings.

> **Interview tip:** Mention `daemon off;` (Nginx must run in the foreground as PID 1) and the non-root image. Both show real experience.

25. Write a multi-stage Dockerfile for a Java application.

Short answer: Stage one uses a Maven + JDK image to build the JAR, with dependencies downloaded in a separate cached layer. Stage two uses only a JRE image, copies the JAR, runs as a non-root user, and starts Java with container-friendly memory settings.

Example

```
# syntax=docker/dockerfile:1

# ---------- Stage 1: Build ----------
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src

COPY pom.xml .
RUN --mount=type=cache,target=/root/.m2 mvn -B dependency:go-offline

COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -B package -DskipTests

# ---------- Stage 2: Runtime ----------
FROM eclipse-temurin:17-jre
WORKDIR /app

RUN groupadd --system app && useradd --system --gid app --uid 10001 app
COPY --from=build --chown=app:app /src/target/*.jar /app/app.jar
USER app

ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0 -XX:+ExitOnOutOfMemoryError"
EXPOSE 8080

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

Important lines

- `# syntax=docker/dockerfile:1` — enables current BuildKit features like cache mounts.

- `--mount=type=cache,target=/root/.m2` — reuses the Maven repository between builds without storing it in the image.
- `COPY pom.xml` before `src` — dependencies are cached until `pom.xml` changes.
- `-DskipTests` — tests already ran in the Jenkins test stage; remove it if the image build is your only build.
- `eclipse-temurin:17-jre` — runtime only, no JDK or Maven.
- `USER app` — non-root user.
- `MaxRAMPercentage=75.0` — heap follows the container memory limit.
- `sh -c "exec java ..."` — lets us use `$JAVA_OPTS`, and `exec` replaces the shell, so Java is still PID 1 and gets `SIGTERM`.
- The `*.jar` glob assumes the build produces exactly one JAR in `target/`.

Build and run

```
docker build -t orders-service:1.0 .
docker run -d -p 8080:8080 --memory=1g orders-service:1.0
```

> **Interview tip:** Explain three things: dependency caching, JRE-only runtime, and non-root user. Those cover performance, size and security.

26. Can we write a Dockerfile without a base image?

Short answer: Every Dockerfile build stage needs a `FROM` instruction, but we can use `FROM scratch`, which is an empty base image with no OS at all. It works for statically compiled binaries, like Go or Rust apps, and produces very small, very secure images. For Java or Python, we need a runtime, so `scratch` isn't practical; we use distroless or slim images instead.

Example (Go app)

```
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /app .

FROM scratch
COPY --from=build /app /app
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
USER 10001
ENTRYPOINT ["/app"]
```

- `CGO_ENABLED=0` makes a static binary with no C library dependency.

- We copy CA certificates so HTTPS calls work.
- The final image may be only 10–20 MB.

Trade-offs of `scratch`: no shell, no package manager, no debugging tools. You can't `docker exec ... sh` into it. That's good for security, but troubleshooting needs other methods (logs, metrics, a debug sidecar or ephemeral container in Kubernetes).

> **Interview tip:** Answer "technically no — `FROM` is required — but `FROM scratch` gives you an empty base." That precise wording shows you know the detail.

27. How do you build a Docker image and run a container from it?

(Also covers source Q33: "How do you build a Docker image?")

Short answer: I run `docker build -t name:tag .` in the folder that has the Dockerfile. Docker sends the build context to the builder, runs each instruction, and tags the image. Then I run it with `docker run -d --name <name> -p host:container name:tag`, and check it with `docker ps` and `docker logs`.

Build

```
docker build -t orders-service:1.0 .
docker build -t orders-service:1.0 -f docker/Dockerfile .         #
Dockerfile in another path
docker build --build-arg APP_VERSION=1.0 -t orders-service:1.0 .  # pass
build args
docker build --no-cache -t orders-service:1.0 .                    # ignore
cache
docker build --platform linux/amd64 -t orders-service:1.0 .       # build for
a target platform
```

- `-t` — name and tag. Always use a version tag, not just `latest`.
- `.` — the build context.

Run and verify

```
docker run -d --name orders -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=dev \
  --memory=1g --restart unless-stopped \
  orders-service:1.0

docker ps                      # is it running?
docker logs -f orders          # application logs
curl http://localhost:8080/actuator/health
```

Push to a registry

```
docker tag orders-service:1.0 123456789012.dkr.ecr.ap-south-
1.amazonaws.com/orders-service:1.0
docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-service:1.0
```

Hands-on experience: Locally, I built and ran images like this to test a Dockerfile change. In CI, Jenkins did the same steps automatically, tagging the image with the Git commit SHA and pushing it to ECR after the Trivy scan.

> **Interview tip:** Mention `-d`, `-p`, `-e`, `--name` and the verification steps. Interviewers want to see the full loop: build → run → check.

Part 5 — Everyday Commands, Logs & Volumes

28. How do you see container logs from the host machine?

Short answer: I use `docker logs <container>`. It shows everything the application writes to stdout and stderr. I add `-f` to follow live, `--tail 100` for the last lines, `--since 30m` for a time window, and `-t` for timestamps. For Compose, `docker compose logs -f <service>`.

Commands

```
docker logs orders                        # all logs
docker logs -f orders                     # follow live (like tail -f)
docker logs --tail 200 orders             # last 200 lines
docker logs --since 30m -t orders         # last 30 minutes, with timestamps
docker logs orders 2>&1 | grep -i error   # search errors
docker compose logs -f app                # Compose service logs
```

Important: `docker logs` only shows what the app writes to **stdout/stderr**. If the app writes to a file inside the container, `docker logs` shows nothing. The fix is to configure the app to log to the console — the standard practice for containers.

Hands-on experience: A Spring Boot app showed empty `docker logs` because its logback config wrote only to `/app/logs/app.log`. We changed it to a console appender. That also made logs flow to our central logging (Fluent Bit → OpenSearch) in Kubernetes.

> **Interview tip:** Mention the stdout/stderr rule. It's the reason containers log differently from VMs.

29. Where are Docker logs stored in Linux?

Short answer: With the default `json-file` logging driver, each container's logs are stored at `/var/lib/docker/containers/<container-id>/<container-id>-json.log`. The Docker daemon's own logs are in the systemd journal: `journalctl -u docker`. If the logging driver is changed, for example to `journald` or `awslogs`, logs go there instead.

Where to look

Log	Location
Container logs (json-file driver)	`/var/lib/docker/containers/<id>/<id>-json.log`
Docker daemon logs	`journalctl -u docker` (or `journalctl -u docker.service -f`)
Container logs with journald driver	`journalctl CONTAINER_NAME=orders`
Cloud drivers (awslogs, gcplogs)	CloudWatch Logs / Cloud Logging

```
docker inspect -f '{{.LogPath}}' orders     # exact log file path
docker info --format '{{.LoggingDriver}}'    # current default driver
```

Log rotation (important in production) — by default json-file logs grow forever and can fill the disk. Set limits in `/etc/docker/daemon.json`:

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

Then `sudo systemctl restart docker`. This applies to **new** containers only; existing containers keep their old settings until recreated.

Hands-on experience: A Jenkins agent ran out of disk. `du -sh /var/lib/docker/containers/*` showed one container with a 40 GB log file. We added `max-size` and `max-file` in `daemon.json`, recreated the containers, and the problem never came back.

> **Interview tip:** Always add log rotation to this answer. It shows production awareness.

30. How do you remove unused Docker containers?

Short answer: `docker container prune` removes all stopped containers. For a broader cleanup, `docker system prune` also removes unused networks, dangling images and build cache; adding `-a` removes all unused images, and `--volumes` removes unused volumes. I use these carefully on servers and schedule them on CI agents.

Commands

```
docker container prune -f                        # all stopped containers
docker container prune -f --filter "until=24h"  # stopped more than 24h ago
docker rm $(docker ps -aq -f status=exited)     # same idea, manual way
docker image prune -a -f                         # unused images
docker builder prune -f                          # build cache
docker system prune -af                          # containers, networks,
images, build cache
docker system prune -af --volumes                # also volumes — careful,
deletes data!
docker system df                                 # see what uses disk space
```

- `docker run --rm` removes the container automatically when it exits — great for one-off jobs.

Hands-on experience: On our Jenkins Docker agents, a nightly job ran `docker system prune -af --filter "until=72h"`. It kept recent images for build cache but removed old ones. We never used `--volumes` there, to avoid deleting needed data.

> **Interview tip:** Warn about `--volumes`. Showing caution with destructive commands is a good signal.

31. Write a command to run an Nginx container.

Short answer: `docker run -d --name web -p 8080:80 nginx:stable-alpine`. It runs Nginx in the background, names the container `web`, and maps port 8080 on the host to port 80 in the container. Then `curl http://localhost:8080` shows the Nginx welcome page.

Variations

```
# Basic
docker run -d --name web -p 8080:80 nginx:stable-alpine

# Serve your own site (read-only bind mount)
docker run -d --name web -p 8080:80 \
  -v "$(pwd)/site:/usr/share/nginx/html:ro" \
  nginx:stable-alpine

# Production-style: restart policy and memory limit
docker run -d --name web -p 80:80 \
  --restart unless-stopped --memory=256m \
  nginx:stable-alpine
```

- `-d` — detached (background).
- `--name web` — easy to refer to later.
- `-p 8080:80` — host:container.
- `:ro` — the container can't change the site files.

> **Interview tip:** Remember the order: **host port first, container port second**.

32. How do you attach a volume to a container?

Short answer: With `-v` or the more explicit `--mount` option in `docker run`. For a named volume: `-v pgdata:/var/lib/postgresql/data`. For a bind mount: `-v /host/path:/container/path`. In Compose, under the service's `volumes:` key. A volume can't be added to an already running container — we recreate the container with the volume.

Commands

```
# Named volume (Docker creates it if it doesn't exist)
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16

# Same thing with --mount (clearer, recommended in scripts)
docker run -d --name db \
  --mount type=volume,source=pgdata,target=/var/lib/postgresql/data \
  postgres:16

# Bind mount, read-only
docker run -d --name web \
  --mount
type=bind,source="$(pwd)"/site,target=/usr/share/nginx/html,readonly \
  nginx:stable-alpine

# tmpfs (in memory)
docker run -d --tmpfs /tmp:size=64m myapp:1.0

# Check mounts of a container
docker inspect -f '{{json .Mounts}}' db
```

- `-v` with a host path that doesn't exist creates an empty folder; `--mount` gives an error instead, which is safer.

Hands-on experience: For our local Postgres, data was in a named volume so it survived container recreation. For local development of the Python API, we bind-mounted the source folder so code changes were reflected without rebuilding the image.

> **Interview tip:** Mention you can't attach a volume to a running container. It shows you understand containers are recreated, not modified.

33. How do you get inside a running container?

Short answer: `docker exec -it <container> sh` (or `bash` if the image has it). This starts a new shell process inside the running container. I use it for debugging, like checking files, environment variables or network connectivity. `docker attach` is different: it connects to the main process's input and output.

Commands

```
docker exec -it orders sh                    # interactive shell
docker exec -it orders bash                  # if bash exists
docker exec orders env                       # run one command
docker exec -u 0 -it orders sh               # as root (for debugging only)
docker exec -it orders sh -c 'wget -qO- http://payments:8080/health'
```

- `-i` keeps input open; `-t` gives a terminal.
- Exiting a `docker exec` shell does **not** stop the container.
- With `docker attach`, pressing Ctrl+C can stop the main process; detach with Ctrl+P then Ctrl+Q.

Minimal images (distroless/scratch) have no shell. Options: check logs, use `docker cp` to copy files out, or run a debug container in the same namespaces:

```
docker run -it --rm --network container:orders --pid container:orders
nicolaka/netshoot
```

> **Interview tip:** Explain `exec` vs `attach`, and how you debug a container that has no shell. That second part is an expert-level detail.

34. What command do you use to delete a container?

Short answer: `docker rm <container>` deletes a stopped container. To delete a running one, I either stop it first with `docker stop`, or use `docker rm -f`, which kills it and removes it. Adding `-v` also removes its anonymous volumes.

Commands

```
docker stop orders && docker rm orders   # graceful stop, then remove
docker rm -f orders                      # force: kill + remove
docker rm -v orders                      # also remove anonymous volumes
docker rm $(docker ps -aq -f status=exited)   # remove all exited containers
```

- `docker rm` removes **containers**; `docker rmi` removes **images**.
- Named volumes are never removed by `docker rm`; use `docker volume rm` for those.

> **Interview tip:** Mention `rm` vs `rmi` and that `-f` skips graceful shutdown. Short and precise.

35. How do you create an image from a running container?

Short answer: With `docker commit <container> <new-image:tag>`. It saves the container's current filesystem changes as a new image. It's useful for quick debugging or saving the state of a container for investigation, but for real builds we always use a Dockerfile, because a committed image is not repeatable or reviewable.

Commands

```
docker commit -m "debug snapshot" -a "devops" orders orders-debug:2026-10-12
docker commit --change 'CMD ["java","-jar","/app/app.jar"]' orders orders-
fixed:1.0
docker images | grep orders
```

- `docker commit` does **not** include data in volumes.
- `docker export` (container filesystem as a tar, flattened, no history) and `docker save` (image with layers, for moving between hosts) are related but different.

Hands-on experience: During an incident, I committed a misbehaving container to `orders-debug:<date>` before restarting it, so we could inspect its files later without keeping production broken. The actual fix still went through the Dockerfile and CI pipeline.

> **Interview tip:** Always add "not for production builds — we use Dockerfiles." That's the answer interviewers expect.

36. What Docker commands do you use regularly?

Short answer: Daily, I use `docker build`, `docker run`, `docker ps`, `docker logs`, `docker exec`, `docker images`, `docker push` and `docker pull`. For troubleshooting, `docker inspect`, `docker stats` and `docker events`. For cleanup, `docker system df` and `docker system prune`. And `docker compose up/down/logs` for local stacks.

Category	Commands
Build and ship	`docker build`, `docker tag`, `docker push`, `docker pull`, `docker login`, `docker buildx build`
Run and manage	`docker run`, `docker ps -a`, `docker stop`, `docker start`, `docker restart`, `docker rm`
Debug	`docker logs -f`, `docker exec -it`, `docker inspect`, `docker stats`, `docker top`, `docker events`, `docker cp`
Images	`docker images`, `docker rmi`, `docker history`, `docker image inspect`
Network and storage	`docker network ls/inspect/create`, `docker volume ls/inspect`
Cleanup	`docker system df`, `docker system prune`, `docker container prune`, `docker image prune`

Category	Commands
Compose	`docker compose up -d`, `docker compose ps`, `docker compose logs -f`, `docker compose down`

Hands-on experience: My typical debug sequence for a failing container was: `docker ps -a` (status and exit code) → `docker logs --tail 100` → `docker inspect` (config, env, mounts, OOMKilled) → `docker exec` to test connectivity → `docker stats` for resource usage.

> **Interview tip:** Group commands by purpose instead of listing them randomly. It sounds organized and experienced.

Part 6 — Troubleshooting, Networking, CI/CD & Security

37. How do you troubleshoot a running container?

Short answer: I follow a fixed order: check the container status and exit code with `docker ps -a`, read the logs with `docker logs`, check the configuration with `docker inspect` (env variables, ports, mounts, health, OOMKilled), check resource usage with `docker stats`, and then go inside with `docker exec` to test files, DNS and connectivity. Once I find the root cause, I fix it in the image or configuration and redeploy.

My step-by-step approach

1. **Status** — `docker ps -a`: is it running, restarting or exited? What is the exit code?
2. **Logs** — `docker logs --tail 200 -t <c>`: application errors, stack traces.
3. **Config** — `docker inspect <c>`: environment variables, command, ports, mounts, network, restart count, `State.OOMKilled`, health status.
4. **Resources** — `docker stats <c>`: CPU and memory near the limit?
5. **Processes** — `docker top <c>`: is the app process actually running?
6. **Inside** — `docker exec -it <c> sh`: check config files, `env`, disk space, and connectivity (`wget` / `curl` to dependencies, `nslookup db`).
7. **Network** — `docker port <c>`, `docker network inspect <net>`: correct port mapping and network?
8. **Events** — `docker events --since 30m`: restarts, OOM events, health status changes.
9. **Host** — disk space (`df -h`), daemon logs (`journalctl -u docker`).
10. **Fix and prevent** — fix the Dockerfile, config or limits; add health checks and alerts.

Common issues I've seen

Symptom	Likely cause
Exits immediately with code 0	Main process ended (for example, a service started in the background instead of foreground)
Exit code 1	Application error — check logs
Exit code 137	OOM kill or `docker kill`
App works inside but not from host	Missing `-p`, or app listening on `127.0.0.1` instead of `0.0.0.0`
Can't reach another container	Not on the same user-defined network, or using IP instead of name
Permission denied on a volume	File ownership doesn't match the container's non-root user

Hands-on experience: A container worked inside (`curl localhost:8080` from `docker exec` was fine) but not from the host. `docker inspect` showed the port was published correctly. The real issue was the app config binding to `127.0.0.1`. Changing it to `0.0.0.0` fixed it.

> **Interview tip:** Speak in a clear order — status, logs, inspect, stats, exec. A structured method matters more than knowing every command.

38. How do you troubleshoot memory leaks in a container?

Short answer: First I confirm it's a leak: memory keeps growing over time and never comes down, often ending in an OOM kill with exit code 137. I watch `docker stats` or Grafana over hours, check `OOMKilled` in `docker inspect`, then look inside the app — for Java, heap dumps and GC logs; for Python or Node, memory profilers. I fix the code, set proper limits, and add alerts.

My approach

1. **Confirm the pattern** — steady growth over time (a leak) vs. a sudden spike (a big request or a wrong limit).

```
docker stats --no-stream orders
docker inspect -f '{{.State.OOMKilled}} {{.State.ExitCode}}
{{.RestartCount}}' orders
```

2. **Check limits are sensible** — a container with no limit can consume the host; a limit that's too small looks like a leak but isn't.

3. **Check runtime settings** — for Java, is the heap sized for the container (`-XX:MaxRAMPercentage`)? Non-heap memory (metaspace, threads, direct buffers) also counts.
4. **Collect evidence from the app:**
   - Java: heap dump on OOM (`-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps` with a volume), or `jcmd 1 GC.heap_dump /dumps/heap.hprof`; analyse with Eclipse MAT. GC logs and JVM metrics (Micrometer → Prometheus).
   - Python: `tracemalloc` or memory profilers. Node: heap snapshots.
5. **Find the cause** — common ones: unbounded caches, objects kept in static collections, unclosed connections or streams, growing queues, thread leaks.
6. **Fix and verify** — release the fix, compare memory graphs before and after.
7. **Prevent** — memory limits, alerts on memory growth (for example over 80% of the limit), load tests before release.

Hands-on experience: One Java service was OOM-killed every two days. Grafana showed heap growing steadily. A heap dump analysed in Eclipse MAT showed a `HashMap` used as a cache with no size limit or expiry. We replaced it with a bounded Caffeine cache, and memory became flat.

> **Interview tip:** Separate "a real leak" from "a wrong limit". Showing you check the limit first is a sign of experience.

39. What are Docker networks? Explain each type.

Short answer: Docker networks control how containers talk to each other and to the outside world. The main drivers are **bridge** (default, single host), **host** (container uses the host's network directly), **none** (no networking), **overlay** (across multiple hosts in Swarm), and **macvlan/ipvlan** (container gets an IP on the physical network). In practice, we mostly use user-defined bridge networks, because they give DNS resolution by container name.

Network types

Driver	What it does	When to use
Default `bridge`	Private network on `docker0`; containers reach each other only by IP	Quick tests
User-defined bridge	Private network with built-in DNS by container name, better isolation	Most single-host apps, Compose (default)

Driver	What it does	When to use
`host`	No network isolation; container uses host ports directly	High-performance networking, monitoring agents (Linux)
`none`	Only a loopback interface	Fully isolated batch jobs
`overlay`	Network spanning multiple Docker hosts	Docker Swarm services
`macvlan` / `ipvlan`	Container gets its own IP on the physical LAN	Legacy apps that need a real network IP

Commands

```
docker network ls
docker network create app-net
docker run -d --name db  --network app-net postgres:16
docker run -d --name api --network app-net -p 8080:8080 api:1.0   # api
reaches db at "db:5432"
docker network connect app-net existing-container
docker network inspect app-net
```

Hands-on experience: Docker Compose created a user-defined network for our local stack, so the API connected to `db` and `redis` by name. On Jenkins agents, we ran integration-test containers in a dedicated network per build to avoid conflicts between parallel builds.

> **Interview tip:** The key difference interviewers look for: **default bridge has no DNS by name; user-defined bridge does.**

40. How does Jenkins integrate with Docker?

Short answer: In three main ways. Jenkins **builds and pushes images** in the pipeline using `docker build` / `docker push` or the Docker Pipeline plugin. Jenkins can **run build steps inside containers** using `agent { docker { image '...' } }`, so tools like Maven come from an image. And Jenkins **agents themselves can be containers**, for example Kubernetes pod agents.

1. Build and push images

```
stage('Build & Push') {
    steps {
        script {
            def img = docker.build("mycompany/orders-
service:${env.BUILD_NUMBER}")
            docker.withRegistry('https://registry.hub.docker.com',
'dockerhub-creds') {
                img.push()
            }
        }
    }
}
```

2. Run steps inside a container

```
pipeline {
    agent {
        docker {
            image 'maven:3.9-eclipse-temurin-17'
            args '-v $HOME/.m2:/root/.m2'
        }
    }
    stages {
        stage('Build') { steps { sh 'mvn -B clean package' } }
    }
}
```

- The `-v $HOME/.m2:/root/.m2` mount keeps the Maven cache between builds.

3. Containers as agents — the Kubernetes plugin starts a pod per build; on Kubernetes, images are often built with Kaniko or BuildKit instead of mounting the Docker socket.

Requirements and good practice

- Docker installed on the agent and the `jenkins` user allowed to use it (or a rootless/daemonless builder).
- Registry credentials in the Jenkins Credentials store; for ECR, use the agent's IAM role.
- Unique image tags (commit SHA or build number), Trivy scan before push, cleanup of old images on agents.

Hands-on experience: Our pipeline built the image, scanned it with Trivy, and pushed it to ECR. Build steps ran inside the official Maven image, so we never installed Maven on agents, and every build used the same tool version.

> **Interview tip:** Mention all three integration styles. Most candidates only mention building images.

41. What is `withDockerRegistry` in Jenkins?

Short answer: `withDockerRegistry` is a step from the Jenkins Docker Pipeline plugin. It logs in to a container registry using a credential stored in Jenkins, runs the commands inside its block with that login, and logs out afterwards. It lets us push or pull private images without writing passwords in the Jenkinsfile.

Example

```
stage('Push Image') {
    steps {
        withDockerRegistry([credentialsId: 'nexus-docker-creds', url:
'https://nexus.example.com:8083']) {
            sh 'docker push nexus.example.com:8083/orders-service:$TAG'
        }
    }
}
```

- `credentialsId` — a "Username with password" credential in Jenkins.
- `url` — registry address; an empty URL means Docker Hub.
- Inside the block, every `docker` command is authenticated.
- `docker.withRegistry(url, credId) { ... }` is the equivalent used inside `script` blocks.
- For AWS ECR, either use the Amazon ECR plugin credential format (`ecr:<region>:<aws-credential-id>`), or skip it and log in with `aws ecr get-login-password` using an IAM role.

Hands-on experience: We pushed images to a Nexus Docker repository with `withDockerRegistry`. Each team had its own folder-scoped credential, so a team could only push to its own repository, and passwords never appeared in logs.

> **Interview tip:** Key phrase: "temporary, scoped login using a Jenkins credential."

42. How do you secure Docker images and containers?

Short answer: I secure both the **image** and the **runtime**. For images: minimal trusted base images, pinned versions, multi-stage builds, no secrets inside, and vulnerability scanning with Trivy in CI. For containers: run as non-root, drop Linux capabilities, read-only

filesystem, no privileged mode, resource limits, and never expose the Docker socket. On top of that, image signing and regular base image updates.

Image security (build time)

- **Minimal, official base images** — distroless, `-slim`, Alpine; fewer packages means fewer vulnerabilities.
- **Pin versions** — a specific tag or even a digest (`@sha256:...`), never just `latest`.
- **Multi-stage builds** — no compilers or build tools in the final image.
- **No secrets in images** — not in `ENV`, `ARG` or copied files; use BuildKit secret mounts at build time and inject secrets at runtime.
- **`.dockerignore`** — keep `.env`, keys and `.git` out of the build context.
- **Scan in CI** — `trivy image --exit-code 1 --severity CRITICAL,HIGH`; fail the build on critical issues.
- **Lint** — `hadolint` for Dockerfile best practices.
- **Sign and verify** — sign images (for example with Cosign) and verify signatures before deploy.
- **Rebuild regularly** — pick up base image security patches.

Container security (runtime)

- **Non-root user** — `USER 10001` in the Dockerfile.
- **Drop capabilities** — `--cap-drop ALL`, add back only what's needed.
- **No privilege escalation** — `--security-opt no-new-privileges`.
- **Read-only root filesystem** — `--read-only` with `--tmpfs /tmp` if the app needs scratch space.
- **Never `--privileged`**, and never mount `/var/run/docker.sock` into app containers — it equals root on the host.
- **Resource limits** — `--memory`, `--cpus`, `--pids-limit`.
- **Keep default seccomp/AppArmor profiles**; consider rootless Docker or user namespace remapping.
- **Network** — publish only the needed ports; use separate networks.
- **Host** — keep Docker Engine and the OS patched; restrict who is in the `docker` group.

Example

```
docker run -d --name orders \
  --user 10001 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --memory=1g --cpus=1 --pids-limit=200 \
  -p 8080:8080 \
  orders-service:1.0
```

Hands-on experience: In my project, the Jenkins pipeline failed any image with CRITICAL vulnerabilities in the Trivy scan. All images ran as non-root, and our Kubernetes policies enforced `runAsNonRoot`, `readOnlyRootFilesystem` and dropped capabilities. When a critical OpenSSL issue was announced, we rebuilt all images on the patched base image and redeployed within a day.

> **Interview tip:** Organize the answer as **build time vs runtime**, and name one tool for each (Trivy for scanning, non-root and dropped capabilities for runtime). A structured answer sounds senior.

Appendix — Source question mapping

All 50 source questions are covered. 42 are unique and answered in their original order; the 8 below were repeats or overlaps and were merged into their first occurrence.

Duplicates merged

Source #	Source question	Merged into
13	Reduce Docker image size?	Q6 (source 6)
14	Difference between COPY and ADD?	Q4 (source 4)
15	Difference between CMD and ENTRYPOINT?	Q5 (source 5)
18	What is Docker and explain its architecture?	Q1 and Q2 (sources 1 and 2)
30	Write a Dockerfile for a simple web application.	Q14 (source 17)
32	How do you reduce image size using multi-stage builds?	Q7 (source 7)

Source #	Source question	Merged into
34	How do you build a Docker image and run a container?	Q27 (source 33)
47	What is a Docker volume?	Q10 (source 10)

Similar questions kept separately (they test different things)

Handbook questions	Why both are kept
Q13 what a Dockerfile is · Q23 Dockerfile instructions	Concept vs. full instruction reference
Q14 web app · Q24 Nginx · Q25 Java multi-stage	Three different Dockerfiles that interviewers ask to write
Q16 sharing OS resources · Q17 resource allocation · Q18 namespaces	Overview vs. cgroups and limits vs. isolation
Q10 volumes · Q32 attaching a volume	Concept vs. commands
Q28 viewing logs · Q29 where logs are stored	Commands vs. storage location and rotation
Q30 removing unused containers · Q34 deleting a container	Bulk cleanup with prune vs. removing one container
Q37 troubleshooting a container · Q38 memory leaks	General method vs. one specific deep-dive

Questions rewritten for clarity

- Source 17, "Write a Dockerfile." → Q14, *Write a Dockerfile for a simple web application.*
- Source 40, "How do you enter inside a running container?" → Q33, *How do you get inside a running container?*
- Source 7, "What is a multi-stage Docker build?" → Q7, *What is a multi-stage Docker build, and how does it reduce image size?*